

Raport z listy 4

Sztuczna inteligencja

Spis treści

Eksploracja danych	4
Polecenie	4
Wstępna analiza zbioru	4
Przegląd atrybutów	6
Analiza atrybutów	7
Wartości NaN atrybutów	22
Analiza celu	22
Profile dla każdego statusu	23
Wnioski	24
Podstawa (baseline) – podział danych, przygotowanie danych, modele klasyfikacyjne ..	25
Polecenie	25
Zarys ogólny wykonania	25
Przetworzenie danych	26
Podział zbioru na zbiór deweloperski i testowy	26
Podział zbioru deweloperskiego i walidacja krzyżowa	27
Moduł przetwarzania wstępnego (preprocesor)	27
Podsumowanie wyników dla modeli	28
Porównanie wyników	29
Krótka teoria do Random Forest	32
Porównanie metod postępowania z brakującymi wartościami	33
Imputacja	33
Interpolacja	33
Dokładność (accuracy) w walidacji krzyżowej	34
Ewaluacja na zbiorze testowym	34
Który model i metodę wybrać?	36
Badanie metod przetwarzania danych	37
Krótka analiza sugerowanych metod	37
Kod	37
Które modele warto przeanalizować?	39
Analiza dla klasyfikatora Bayesa	40
Analiza dla Support Vector Machine (SVM)	41

Hiperparametryzacja	43
Klasyfikator Bayesa	43
Drzewo decyzyjne	44
Random Forest	44
SVM	45
Zastosowane metody ładogdzenia przeuczenia	46
Biblioteki	47
Bibliografia.....	47

Eksploracja danych

Polecenie

1. Eksploracja danych — Przedstaw podstawowe dane statystyczne i uwagi dotyczące cech i etykiet zbioru danych. (10 punktów)

Wstępna analiza zbioru

Sprawdzamy najpierw metadane pobranego zbioru danych

- Temat – „Przewidywanie Przeżywalności Pacjentów z Marskością Wątroby”

```
'name': 'Cirrhosis Patient Survival Prediction',
```

- Obszar – Zdrowie i Medycyna

```
'area': 'Health and Medicine',
```

- Cel – przewidywanie stanu przeżycia (survival state) pacjentów z marskością wątroby.

```
'utilize 17 clinical features for predicting survival state of patients with liver cirrhosis.'
```

- Możliwe stany

- 0 = D (śmierć)
- 1 = C (ocenzurowane, inne niż śmierć)
- 2 = CL (ocenzurowane z powodu przeszczepu wątroby)

```
The survival states include 0 = D (death), 1 = C (censored), 2 = CL (censored due to liver transplantation).',
```

- Problem – klasyfikacja

```
'tasks': ['Classification'],
```

- Liczba pacjentów - 418

```
'num_instances': 418,
```

- Liczba cech – 17

```
'num_features': 17,
```

- Typy cech – numeryczne i kategoriowe

```
'feature_types': ['Real', 'Categorical'],
```

- Cechy demograficzne – wiek i płeć (są to też dane wrażliwe)

```
'demographics': ['Age', 'Sex'],
```

```
'sensitive_data': 'Gender, Age',
```

- Kolumna docelowa (y) – Target

```
'target_col': ['Status'],
```

- Mamy brakujące wartości - NaN

```
'has_missing_values': 'yes',
```

```
'missing_values_symbol': 'NaN',
```

- Autorzy

```
'creators': ['E. Dickson',
             'P. Grambsch',
             'T. Fleming',
             'L. Fisher',
             'A. Langworthy'],
```

- Artykuł wprowadzający (intro paper)

```
intro_paper': {'ID': 296,
               'type': 'NATIVE',
               'title': 'Prognosis in primary biliary cirrhosis: Model for decision making',
               'authors': 'E. Dickson, P. Grambsch, T. Fleming, L. Fisher, A. Langworthy',
               'venue': 'Hepatology',
               'year': 1989,
```

- Dodatkowe informacje:

W latach 1974-1984 do May Clinic zgłosiło 424 pacjentów z PBC kwalifikujących się do badania skuteczności D-penicylaminy

- 312 pacjentów zgodziło się udzielić kompletnych informacji i poddać klinicznym próbom
- 106 zgodziło się na pobranie podstawowych informacji i śledzenie przeżywalności
- Z sześcioma pacjentami stracono kontakt

W latach 1974-1984 do May Clinic zgłosiło 424 pacjentów z PBC kwalifikujących się do badania skuteczności D-penicylaminy

Przegląd atrybutów

	name	role	type	demographic	description	units	missing_values
0	ID	ID	Integer	NaN	unique identifier	NaN	no
1	N_Days	Other	Integer	NaN	number of days between registration and the ea...	NaN	no
2	Status	Target	Categorical	NaN	status of the patient C (censored), CL (censor...	NaN	no
3	Drug	Feature	Categorical	NaN	type of drug D-penicillamine or placebo	NaN	yes
4	Age	Feature	Integer	Age	age	days	no
5	Sex	Feature	Categorical	Sex	M (male) or F (female)	NaN	no
6	Ascites	Feature	Categorical	NaN	presence of ascites N (No) or Y (Yes)	NaN	yes
7	Hepatomegaly	Feature	Categorical	NaN	presence of hepatomegaly N (No) or Y (Yes)	NaN	yes
8	Spiders	Feature	Categorical	NaN	presence of spiders N (No) or Y (Yes)	NaN	yes
9	Edema	Feature	Categorical	NaN	presence of edema N (no edema and no diuretic ...	NaN	no
10	Bilirubin	Feature	Continuous	NaN	serum bilirubin	mg/dl	no
11	Cholesterol	Feature	Integer	NaN	serum cholesterol	mg/dl	yes
12	Albumin	Feature	Continuous	NaN	albumin	gm/dl	no
13	Copper	Feature	Integer	NaN	urine copper	ug/day	yes
14	Alk_Phos	Feature	Continuous	NaN	alkaline phosphatase	U/liter	yes
15	SGOT	Feature	Continuous	NaN	SGOT	U/ml	yes
16	Tryglicerides	Feature	Integer	NaN	tryglicerides	NaN	yes
17	Platelets	Feature	Integer	NaN	platelets per cubic	ml/1000	yes
18	Prothrombin	Feature	Continuous	NaN	prothrombin time	s	yes
19	Stage	Feature	Categorical	NaN	histologic stage of disease (1, 2, 3, or 4)	NaN	yes

Analiza atrybutów

1. Drug

- Opis: Rodzaj podawanego leku (D-penicylamina)
- Typ: Kategoryczny
- Brakujące wartości – Tak

3	Drug	Feature	Categorical	NaN	type of drug D-penicillamine or placebo	NaN	yes
---	------	---------	-------------	-----	---	-----	-----

Drug: type of drug D-penicillamine or placebo

```
===== Drug =====  
  
unique: <StringArray>  
['D-penicillamine', 'Placebo', 'NaN']  
Length: 3, dtype: str  
  
counts:  
  Drug  
D-penicillamine    158  
Placebo            154  
NaN                105  
NaN                1  
Name: count, dtype: int64
```

Co ciekawe aż połowie pacjentów podano placebo. 106 pacjentów (25%) zgodnie z opisem nie poddało się badaniom klinicznym (NaN). Przed kolejnymi etapami będzie trzeba przekonwertować NaN na NaN (literówka?)

2. Age

- Opis: Wiek pacjenta w dniach
- Typ: Numeryczny
- Brakujące wartości – Nie

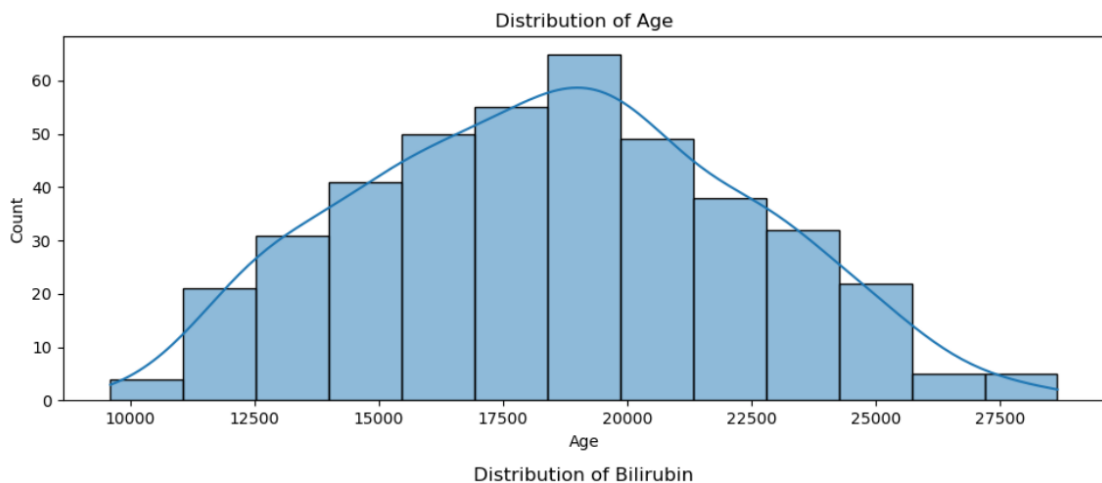
4	Age	Feature	Integer	Age	age	days	no
---	-----	---------	---------	-----	-----	------	----

5. Age: age in [days]					
	min	mean	median	max	std
Age	9598.00	18533.351675	18628.00	28650.00	3815.845055

Wiek jest podany w dniach życia, konwertując na lata mamy:

- Minimalnie: 26.3
- Średnia: 50.7
- Mediana: 51.0
- Maksymalnie: 78.4
- Standardowe odchylenie: 10.4

Wykres



Spodziewałem się, że najwięcej pacjentów będzie w wieku starszym, a wykres sugeruje, że pacjentów w wieku poniżej 50 lat może być więcej niż tych w wieku powyżej 50 lat. Średni wiek pacjentów to ok. 50-55 lat.

3. Sex

- Opis: Dwuklasowa klasyfikacja biologicznej płci pacjenta
- Typ: Kategoryczny
- Brakujące wartości – Nie

5	Sex	Feature	Categorical	Sex	M (male) or F (female)	NaN	no
---	-----	---------	-------------	-----	------------------------	-----	----

6. Sex: M (male) or F (female)

```
===== Sex =====  
  
unique: <StringArray>  
['F', 'M']  
Length: 2, dtype: str  
  
counts:  
  Sex  
F    374  
M     44  
Name: count, dtype: int64
```

Około 89% pacjentów to kobiety.

4. Ascites

- Opis: Obecność wodobrzusza (nieprawidłowe nagromadzenie płynu w jamie brzusznej)
- Typ: Kategoryczny
- Brakujące wartości – Tak

6	Ascites	Feature	Categorical	NaN	presence of ascites N (No) or Y (Yes)	NaN	yes
---	---------	---------	-------------	-----	---------------------------------------	-----	-----

7. Ascites: presence of ascites N (No) or Y (Yes)

```
===== Ascites =====  
  
unique: <StringArray>  
['Y', 'N', 'NaN']  
Length: 3, dtype: str  
  
counts:  
  Ascites  
N        288  
NaN      105  
Y         24  
NaN         1  
Name: count, dtype: int64
```

Tylko niecałe 8% pacjentów poddanych próbie klinicznej miało wodobrzusze.

5. Hepatomegaly

- Opis: Obecność hepatomegalii (powiększenie wątroby)
- Typ: Kategoryczny
- Brakujące wartości – Tak

7	Hepatomegaly	Feature	Categorical	NaN	presence of hepatomegaly N (No) or Y (Yes)	NaN	yes
---	--------------	---------	-------------	-----	--	-----	-----

8. Hepatomegaly: presence of hepatomegaly N (No) or Y (Yes)

```
===== Hepatomegaly =====  
  
unique: <StringArray>  
['Y', 'N', 'NaN']  
Length: 3, dtype: str  
  
counts:  
  Hepatomegaly  
Y           160  
N           152  
NaN          105  
NaNN           1  
Name: count, dtype: int64
```

Okolo 51% pacjentów ma powiększoną wątrobę

6. Spiders

- Opis: Obecność naczynek gwiazdzistych (zmiany skórne w postaci centralnej, jasnoczerwonej plamki z drobnymi, promieniste odchodzącymi naczyniami przypominającymi nogi pająka)
- Typ: Kategoryczny
- Brakujące wartości – Tak

8	Spiders	Feature	Categorical	NaN	presence of spiders N (No) or Y (Yes)	NaN	yes
---	---------	---------	-------------	-----	---------------------------------------	-----	-----

9. Spiders: presence of spiders N (No) or Y (Yes)

```
===== Spiders =====  
  
unique: <StringArray>  
['Y', 'N', 'NaN']  
Length: 3, dtype: str  
  
counts:  
  Spiders  
N          222  
NaN         105  
Y           90  
NaNN          1  
Name: count, dtype: int64
```

Okolo 28% pacjentów ma naczyniaki gwiazdziste

7. Edema

- Opis: Obecność obrzęku (opuchlizna spowodowana nadmiarem płynu zatrzymanego w tkankach), możliwe wartości: N (brak obrzęku i brak terapii diuretycznej), S (brak obrzęku po terapii diuretycznej lub obrzęk bez terapii diuretycznej), Y (obrzęk mimo terapii diuretycznej)
- Typ: Kategoryczny
- Brakujące wartości – Nie

```
9      Edema  Feature  Categorical  NaN  presence of edema N (no edema and no diuretic ...  NaN  no
Edema: presence of edema N (no edema and no diuretic therapy for edema), S (edema present without diuretics, or
or edema resolved by diuretics), or Y (edema despite diuretic therapy)\n1

===== Edema =====
unique: <StringArray>
['Y', 'N', 'S']
Length: 3, dtype: str

counts:
  Edema
N      354
S       44
Y       20
Name: count, dtype: int64
```

Okolo 85% nie miało obrzęku i nie byli poddani terapii diuretycznej. Okolo 5% miało obrzęki pomimo terapii diuretycznej. Pozostali pacjenci albo mieli obrzęk bez terapii albo obrzęk ustąpił po terapii.

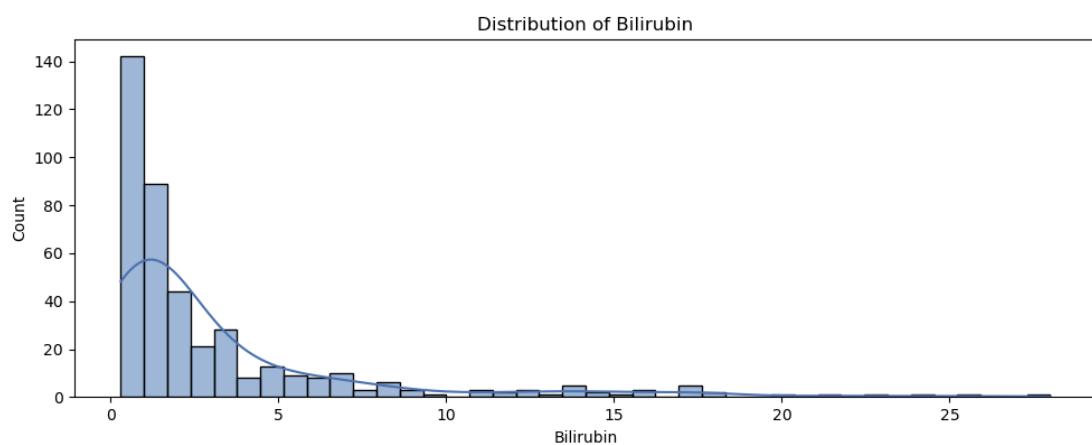
8. Bilirubin

- Opis: Poziom bilirubiny mierzony w mg/dl
- Typ: Numeryczny
- Brakujące wartości – Nie

10	Bilirubin	Feature	Continuous	NaN	serum bilirubin	mg/dl	no
----	-----------	---------	------------	-----	-----------------	-------	----

11. Bilirubin: serum bilirubin in [mg/dl]\

	min	mean	median	max	std
Bilirubin	0.30	3.220813	1.40	28.00	4.407506



Większość pacjentów miała bardzo niski poziom bilirubiny (połowa poniżej 3.2 mg/dl). Jest to rozkład o prawostronnej skośności (right-skewed distribution)

9. Cholesterol

- Opis: Poziom cholesterolu mierzony w mg/dl
- Typ: Numeryczny
- Brakujące wartości – Yes

11	Cholesterol	Feature	Integer	NaN	serum cholesterol	mg/dl	yes
----	-------------	---------	---------	-----	-------------------	-------	-----

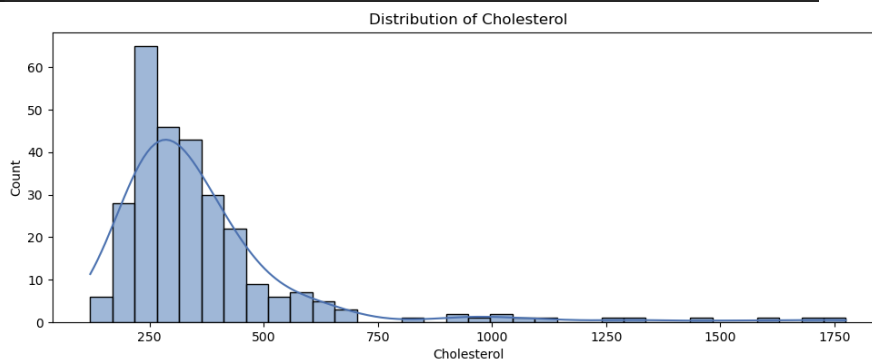
12. Cholesterol: serum cholesterol in [mg/dl]

```
===== Cholesterol =====
unique: <StringArray>
[ '261', '302', '176', '244', '279', '248', '322', '280', '562',
  '200',
  ...
  '194', '1000', '646', '275', '340', '342', '393', '335', '434',
  '576']
Length: 202, dtype: str

counts:
Cholesterol
NaN      106
NaNN     28
260       4
316       4
448       4
...
342       1
393       1
335       1
434       1
576       1
Name: count, Length: 203, dtype: int64
```

Rozkład cholesterolu jest numeryczny, ale podejrzewam, że dane są nieczyste (obecność NaNN), dlatego wskazało, że cholesterol jest kategoriyczny. Trzeba zatem przekonwertować tę kolumnę na numeryczną.

	min	mean	median	max	std
Cholesterol	120.00	369.510563	309.50	1775.00	231.944545



Wykres pokazuje, że większość badanych osób ma cholesterol w dolnych i średnich przedziałach prezentowanej skali (centrum skupione wokół 250–300), jednak w zbiorze danych występuje nieliczna grupa pacjentów o nietypowo wysokich, wręcz skrajnych wartościach, które mocno zaburzają symetrię rozkładu.

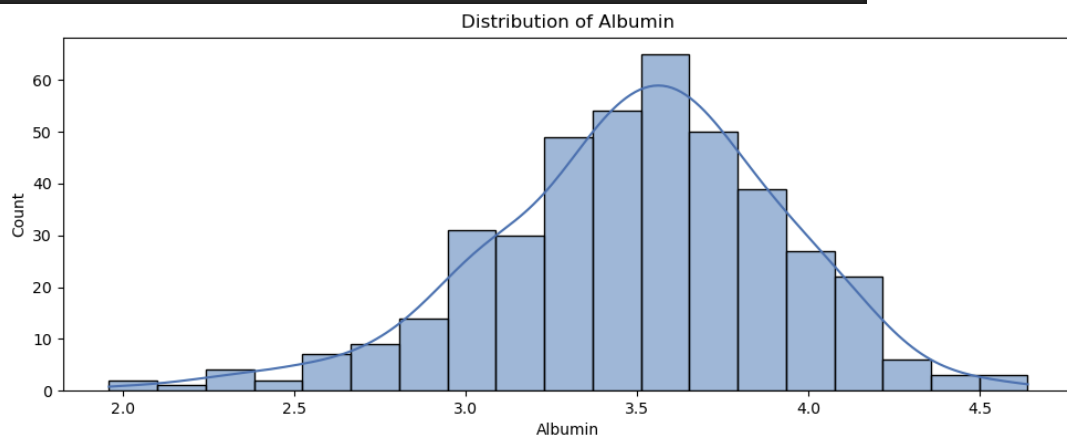
10. Albumin

- Opis: Poziom albuminu mierzony w gm/dl
- Typ: Numeryczny
- Brakujące wartości – Nie

12	Albumin	Feature	Continuous	NaN	albumin	gm/dl	no
----	---------	---------	------------	-----	---------	-------	----

13. Albumin: albumin in [gm/dl]

	min	mean	median	max	std
Albumin	1.96	3.497440	3.53	4.64	0.424972



Rozkład jest zbliżony do normalnego, ale ma lekką skośność lewostronną.

11. Copper

- Opis: Poziom miedzi w moczu mierzony w ug/day
- Typ: Numeryczny
- Brakujące wartości – Tak

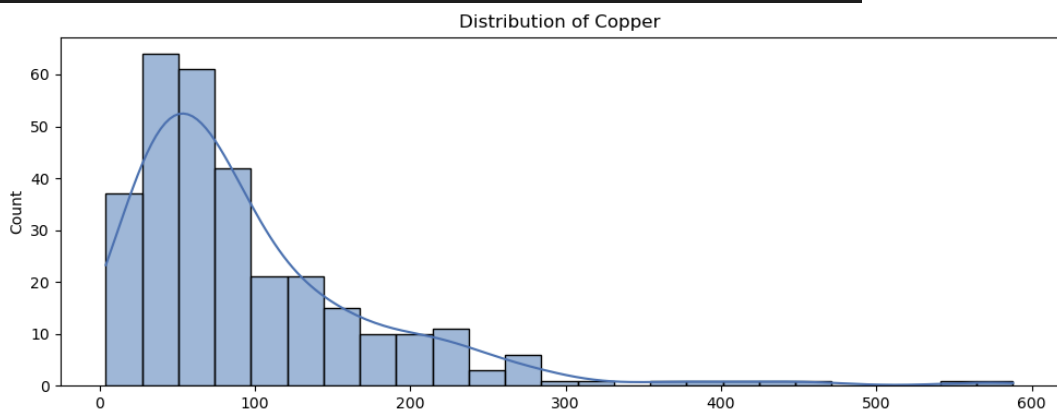
13	Copper	Feature	Integer	NaN	urine copper	ug/day	yes
----	--------	---------	---------	-----	--------------	--------	-----

14. Copper: urine copper in [ug/day]

```
===== Copper =====  
  
unique: <StringArray>  
['156', '54', '210', '64', '143', '50', '52', '79', '140', '46',  
...  
'97', '70', '155', '107', '177', '196', '100', '90', '234', '186']  
Length: 159, dtype: str  
  
counts:  
Copper  
NaN      106  
52         8  
67         7  
58         6  
75         6  
...  
196         1  
100         1  
90          1  
234         1  
186         1  
Name: count, Length: 160, dtype: int64
```

Sytuacja taka sama jak z cholesterolem. Jest to typ numeryczny, należy przekonwertować 'NaN' na NaN w bibliotece Numpy

	min	mean	median	max	std
Copper	4.00	97.648387	73.00	588.00	85.613920



Rozkład poziomu miedzi w moczu jest prawostronnie skośny (right-skewed distribution)

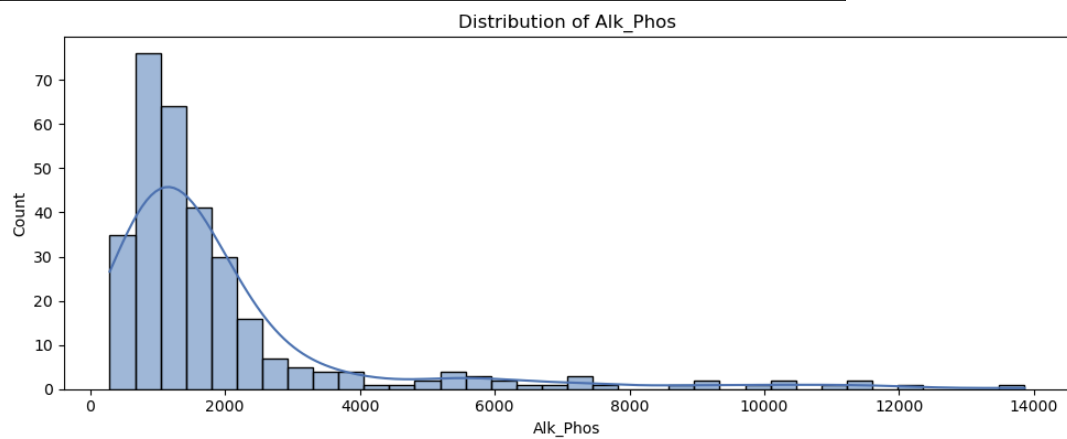
12. Alk_Phos

- Opis: Poziom fosfatazy alkalicznej mierzony w U/liter
- Typ: Numeryczny
- Brakujące wartości – Tak

14	Alk_Phos	Feature	Continuous	NaN	alkaline phosphatase	U/liter	yes
----	----------	---------	------------	-----	----------------------	---------	-----

15. Alk_Phos: alkaline phosphatase in [U/liter]

	min	mean	median	max	std
Alk_Phos	289.00	1982.655769	1259.00	13862.40	2140.388824



Rozkład poziomu fosfatazy alkalicznej jest prawostronnie skośny (right-skewed distribution)

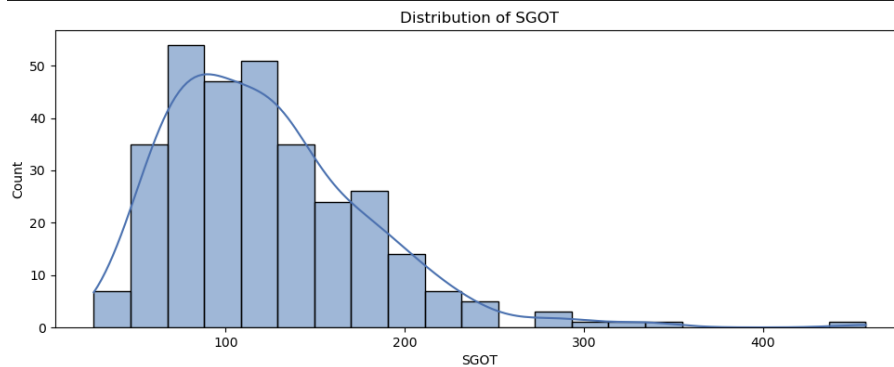
13. SGOT

- Opis: Poziom aminotransferazy asparaginianowej mierzony w U/ml
- Typ: Numeryczny
- Brakujące wartości – Tak

15	SGOT	Feature	Continuous	NaN	SGOT	U/ml	yes
----	------	---------	------------	-----	------	------	-----

16. SGOT: SGOT in [U/ml]

	min	mean	median	max	std
SGOT	26.35	122.556346	114.70	457.25	56.699525



Rozkład poziomu aminotransferaza asparaginianowa jest prawostronnie skośny (right-skewed distribution)

14. Triglycerides

- Opis: Liczba triglicerydów
- Typ: Numeryczny
- Brakujące wartości – Tak

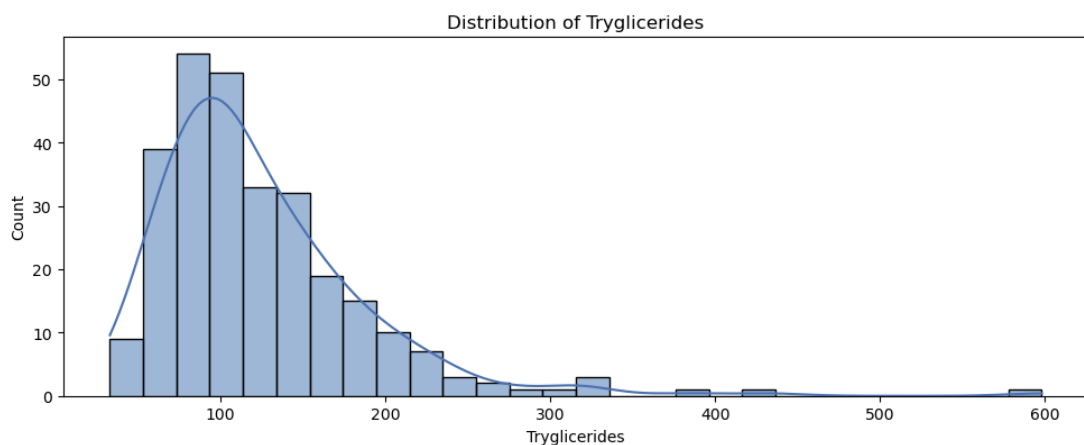
16	Tryglicerides	Feature	Integer	NaN	tryglicerides	NaN	yes
----	---------------	---------	---------	-----	---------------	-----	-----

17. Triglycerides: triglycerides in [mg/dl]

```
===== Tryglicerides =====  
  
unique: <StringArray>  
['172', '88', '55', '92', '72', '63', '213', '189', '143', '79',  
...  
'116', '193', '229', '169', '242', '153', '127', '144', '59', '82']  
Length: 147, dtype: str  
  
counts:  
Tryglicerides  
NaN      106  
NaNNN     30  
118        7  
91         6  
90         6  
...  
153        1  
127        1  
144        1  
59         1  
82         1  
Name: count, Length: 148, dtype: int64
```

Sytuacja taka sama jak z cholesterolem i miedzią w moczu, jest to typ numeryczny ale należy przekonwertować 'NaN' na NaN z biblioteki Numpy

	min	mean	median	max	std
Tryglicerides	33.00	124.702128	108.00	598.00	65.148639



Rozkład liczby triglicerydów jest prawostronnie skośny (right-skewed distribution)

15. Platelets

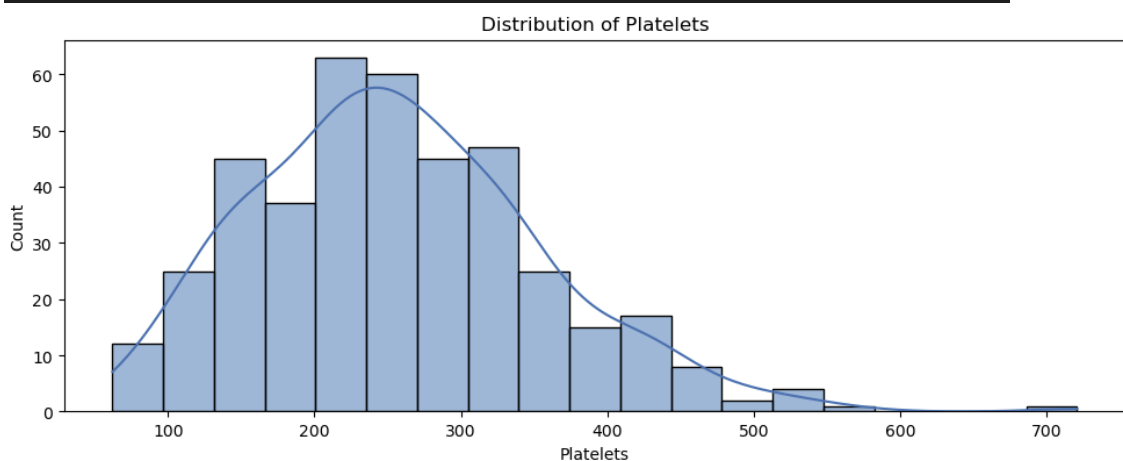
- Opis: Liczba płytek krwi na jednostkę objętości (ml/1000)
- Typ: Numeryczny
- Brakujące wartości – Tak

17	Platelets	Feature	Integer	NaN	platelets per cubic	ml/1000	yes
----	-----------	---------	---------	-----	---------------------	---------	-----

```
18. Platelets: platelets per cubic [ml/1000]\n\n==== Platelets ====\n\nunique: <StringArray>\n[ '190', '221', '151', '183', '136', 'NaN', '204', '373', '251',\n  '302',\n  ...\n  '120', '402', '425', '189', '193', '219', '220', '162', '174',\n  '350']\nLength: 244, dtype: str\n\ncounts:\nPlatelets\n...\n162    1\n174    1\n350    1\nName: count, Length: 245, dtype: int64
```

Sytuacja taka sama jak z cholesterolem, miedzią w moczu i liczbą triglicerydów, jest to typ numeryczny ale należy przekonwertować 'NaN' na NaN z biblioteki Numpy

	min	mean	median	max	std
Platelets	62.00	257.024570	251.00	721.00	98.325585



Rozkład liczby płytek krwi jest prawostronnie skośny (right-skewed distribution), choć niedaleko mu do przybliżenia rozkładu normalnego.

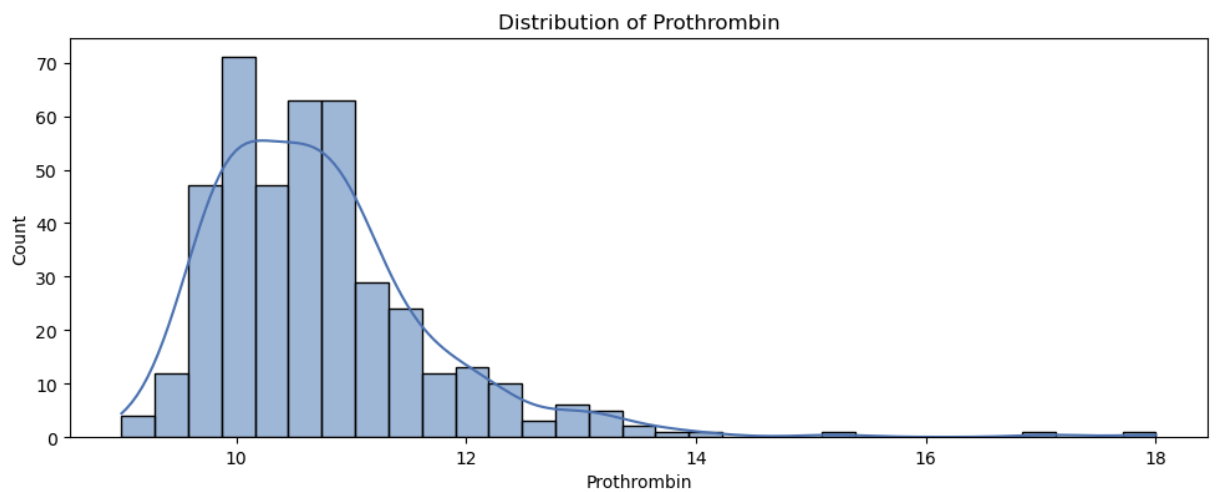
16. Prothrombin

- Opis: Czas protrombinowy (miara czasu krzepnięcia osocza), mierzony w sekundach
- Typ: Numeryczny
- Brakujące wartości – Tak

18	Prothrombin	Feature	Continuous	NaN	prothrombin time	s	yes
----	-------------	---------	------------	-----	------------------	---	-----

19. Prothrombin: prothrombin time in seconds [s]

	min	mean	median	max	std
Prothrombin	9.00	10.731731	10.60	18.00	1.022000



Rozkład czasu protrombinowego jest prawostronnie skośny (right-skewed distribution),

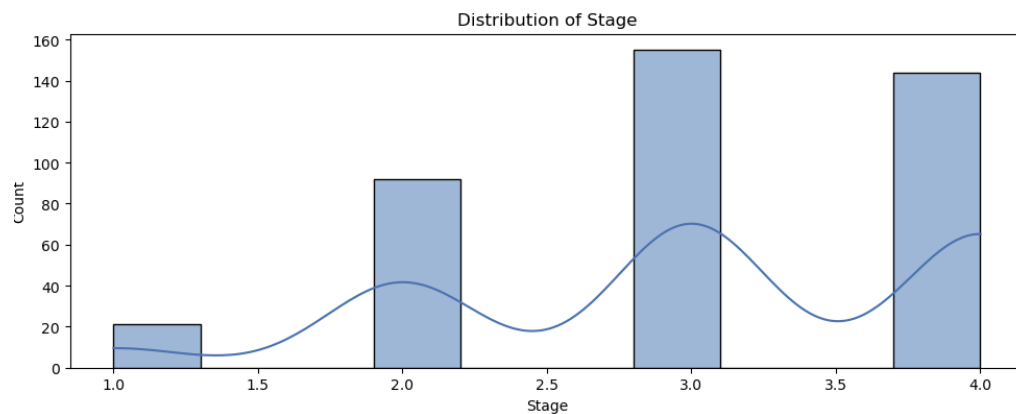
17. Stage

- Opis: Histologiczne stadium choroby (1, 2, 3, 4)
- Typ: Kategoryczny
- Brakujące wartości – Nie

19	Stage	Feature	Categorical	NaN	histologic stage of disease (1, 2, 3, or 4)	NaN	yes
----	-------	---------	-------------	-----	---	-----	-----

Stage: histologic stage of disease (1, 2, 3, or 4)

	min	mean	median	max	std
Stage	1.00	3.024272	3.00	4.00	0.882042



Trzeba to zamienić na typ kategoryczny

```
counts:
  Stage
  3.0    155
  4.0    144
  2.0     92
  1.0     21
  NaN      6
  Name: count, dtype: int64
```

Najwięcej jest pacjentów z 3 albo 4 stadium, najmniej z 1.

Wartości NaN atrybutów

Tryglicerides	136
Cholesterol	134
Copper	108
Alk_Phos	106
SGOT	106
Drug	105
Hepatomegaly	105
Spiders	105
Ascites	105
Platelets	11
Stage	6
Prothrombin	2
Age	0
Bilirubin	0
Edema	0
Sex	0
Albumin	0

Analiza celu

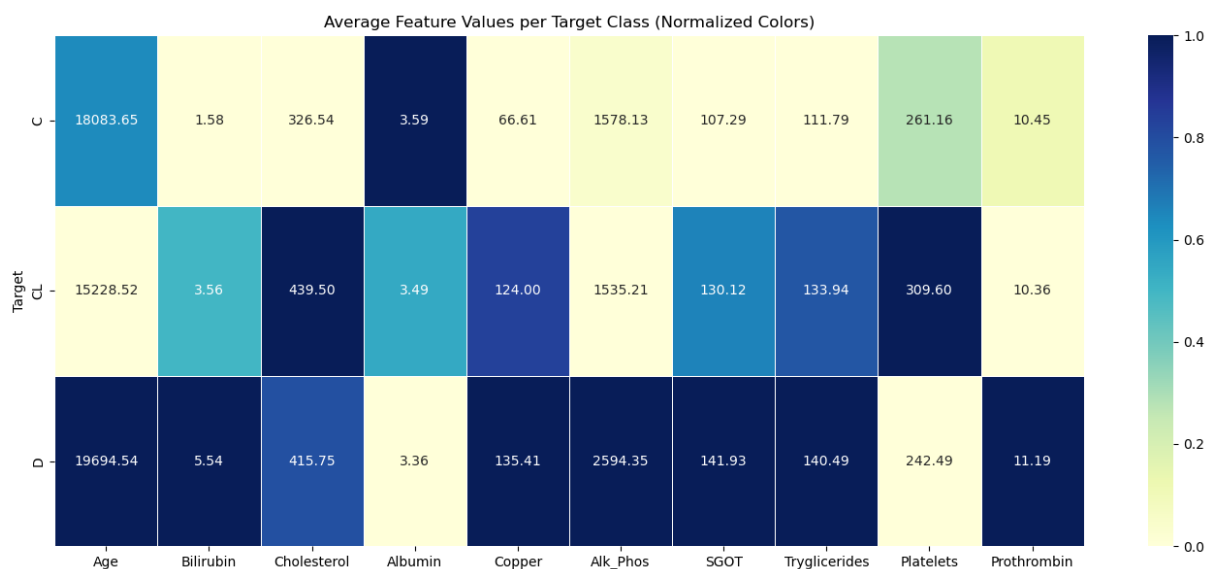
Jest to status pacjenta

Status	
C	232
D	161
CL	25
Name: count, dtype: int64	

38.5% pacjentów zginęło, 6% pacjentów miało przeszczep wątroby, kolejne 55.5% osób przeżyło a ich dalszy stan zdrowia został oceniany

Profile dla każdego statusu

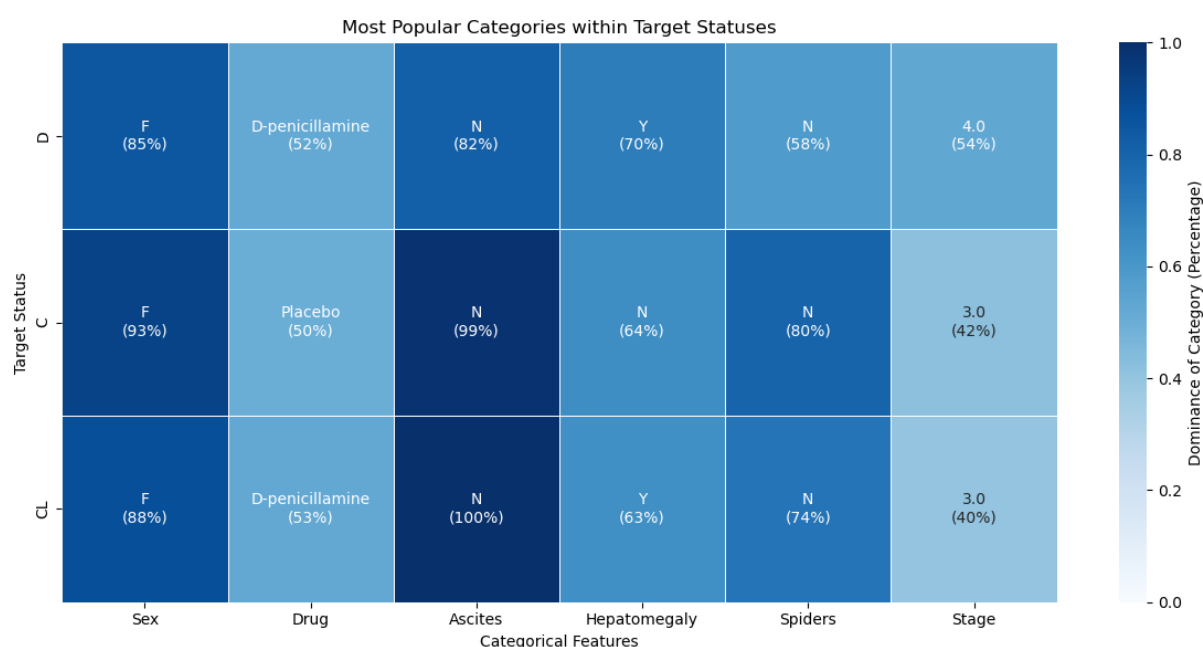
W celach wizualizacji profilu pacjentów dla każdego rezultatu policzyłem średnią każdego atrybutu numerycznego.



Można wywnioskować ciekawe zależności

- Pacjenci, którzy zmarli (średnio 54 lata) byli średnio o 4.5 lat starsi od pacjentów, którzy przeżyli (średnio 49.5 lat)
- Im młodszy pacjent jest, tym jest większe prawdopodobieństwo, że będzie zakwalifikowany do przeszczepu (średnio 41.7 lat)
- Pacjenci, którzy zmarli mają największe poziomy
 - Bilirubiny
 - Miedzi w moczu
 - Fosfotazy alkalicznej
 - Aminotransferazy asparaginianowej
 - Liczby płytek triglicerydów
 - Czasy protrombinowe
- Im większa liczba płytek krwi i poziom albuminy tym większa szansa na przeżycie i przeszczep

Dla kategorii zrobiłem wykres najpopularniejszych kategorii dla każdego rezultatu



Pacjenci, co zmarli, mają częściej obecność wodobrzusza, obecność heptomegalii (powiększenie wątroby) i rzadziej obecność naczynek gwiazdzystych. Większość zmarłych pacjentów było w 4 stadium.

Wnioski

Status pacjenta prawdopodobnie może być wywnioskowany ze sporą dokładnością, ponieważ występują pomiędzy nimi różnice. Zatem stworzenie odpowiedniego modelu powinno być możliwe, jedyną moją obawą jest niewielka ilość danych (tylko 418 pacjentów, z których 312 poddało się badaniom klinicznym)

Podstawa (baseline) – podział danych, przygotowanie danych, modele klasyfikacyjne

Polecenie

2. Przygotowanie danych — Podziel dane na zestaw uczący i walidacyjny (alternatywnie użyj walidacji krzyżowej). Przeprowadź wstępne przetwarzanie danych: (i) Zastosuj jedną lub więcej metod postępowania z brakującymi wartościami (sugerowane: usuwanie, imputacja, interpolacja), aby zachować jak najwięcej informacji o znaczeniu analitycznym; (ii) Zbadaj wpływ różnych metod przetwarzania danych (sugerowane: normalizacja, standaryzacja, dyskretyzacja, selekcja cech, PCA) na wyniki klasyfikacji, porównując wyniki bez przetwarzania danych z wynikami po przetworzeniu danych przy użyciu co najmniej 2 różnych metod (osobno). (30 punktów)
3. Klasyfikacja — Przeprowadź eksperymenty z wykorzystaniem tych klasyfikatorów: (1) naiwny klasyfikator Bayesa oraz (2) drzewo decyzyjne. Wykonaj dostrojenie hiperparametrów dla każdego algorytmu, stosując co najmniej 3 różne zestawy hiperparametrów. Zbadaj wpływ tych zmian na wyniki klasyfikacji.
Bonus — Przetestuj (ze zrozumieniem!) bardziej zaawansowane algorytmy, takie jak Random Forest czy Support Vector Machine (SVM). (5 punktów)
Bonus — W odniesieniu do jednego wybranego klasyfikatora zaproponuj jedno podejście mające na celu złagodzenie przeuczenia. Zbadaj wpływ tego podejścia na wyniki klasyfikacji. (5 punktów)
4. Ewaluacja klasyfikacji — Do porównania wyników różnego typu przygotowania danych oraz wykorzystanego klasyfikatora użyj poznanych metryk oceny klasyfikacji i zinterpretuj wyniki. (20 punktów)

Zarys ogólny wykonania

Uznałem, że stworzę bazowego notebooka w Python, w którym

1. Przetworzę dane
2. Podzielę dane na zbiór testowy i zbiór deweloperski
3. Podzielę zbiór deweloperski na zbiory treningowe i testowe w walidacji krzyżowej
4. Stworzę moduł przetwarzania wstępnego (preprocesor) obsługujący brakujące dane i posiadający kodowanie wektorowe typu jeden z n (splitter) do rozbicia typów kategoriycznych
5. Zrobię zestawienie dokładności każdego folda i raport klasyfikacji dla następujących algorytmów
 - Klasyfikator Bayesa
 - Drzewo decyzyjne
 - Random Forest
 - Support Vector Machine

W następnych sekcjach omówię w szczegółach kolejne sekcje

Przetworzenie danych

```
# NaNN and NaN --> np.nan
X = X.replace(["NaN", "NaNN", "", " "], np.nan)

# Change categorical variables to numeric
cols_to_numeric = ["Cholesterol", "Copper", "Tryglicerides", "Platelets"]
X[cols_to_numeric] = X[cols_to_numeric].apply(pd.to_numeric, errors="coerce")

# Change Stage to category
X["Stage"] = X["Stage"].astype("category")

# Drop rows with missing attributes
X = X.dropna()
y = y.loc[X.index]
```

Tutaj zastępujemy ciągi znaków „NaN” i „NaNN” pustymi typami biblioteki Numpy. Następnie zmieniamy typ na numeryczny wszystkich atrybutów, które zostały nieślusownie zaklasyfikowane jako katagoryczne. Zmieniamy również typ stadium choroby na katagoryczny ponieważ stadia (1, 2, 3, 4) nie są liczbami rzeczywistymi.

Ostatecznie na końcu usuwamy wszystkie puste wartości – przy dalszych analizach te linijki będą usuwane w przypadku testowania technik obsługi pustych danych.

Podział zbioru na zbiór deweloperski i testowy

```
# Split data into training and test sets
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, random_state=67, stratify=y)
```

Zbiór testowy będzie liczył 20% wierszy (84 pacjentów). Flaga stratify oznacza równe proporcje klas w zbiorze deweloperskim i testowym.

Podział zbioru deweloperskiego i walidacja krzyżowa

```
# Cross validation strategy
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=67)
```

Do testów będę stosował walidację krzyżową. Walidacja krzyżowa polega na podziale zbioru danych na kilka równych części, z których kolejno jedna służy jako zestaw testowy, a pozostałe jako treningowy. Proces ten jest powtarzany wielokrotnie, dzięki czemu każdy podzbiór zostanie dokładnie raz użyty do oceny modelu, co pozwala na stabilne i wiarygodne oszacowanie jego skuteczności.

Trudność sprawiło tutaj dobranie odpowiedniego k (n_splits), ponieważ w zbiorze deweloperskim mamy tylko 334 pacjentów (z czego tylko 250 co poddali się badaniom klinicznym). Podział na 5 części oznaczałby zestawy treningowe mające po 267 pacjentów (200 co poddało się badaniom klinicznym). Z kolei dzieląc na 10 części mielibyśmy zestawy po 300 pacjentów (225 pacjentów co poddało się badaniom klinicznym). Przy tak małej ilości danych stawiam na jak największy zestaw treningowy.

Moduł przetwarzania wstępnego (preprocesor)

```
# Preprocessor
cat_cols = X_rest.select_dtypes(include=["object", "str", "category"]).columns
num_cols = X_rest.select_dtypes(include=["number"]).columns

preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), cat_cols),
    ]
)
```

Na chwilę obecną w preprocesorze zawarłem jedynie kodowanie wektorowe typu jeden z n (splitter), który polega na rozbiciu typów kategoriycznych na kilka kolumn. Każda kolumna jest typem bool (0 albo 1) dla konkretnej kategorii w ramach atrybutu.

W przyszłości zamieścimy w preprocesorze zastępowanie pustych wartości, na razie wiersze z pustymi wartościami zostały usunięte wcześniej.

Podsumowanie wyników dla modeli

Dla każdego z modeli

- Klasyfikator Bayesa
- Drzewo decyzyjne
- Random Forest
- Support Vector Machine

Porównuję średnią dokładności (accuracy) dla zestawów treningowych w każdej części z walidacji krzyżowej, a także raport klasyfikacji, zawierający

- Dokładność (accuracy)
- Czulość (recall)
- Precyzję (precision)
- F1-score

Kod dla jednego z tych modeli wygląda następująco

```
from sklearn.model_selection import cross_val_score
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report

model = Pipeline([
    ("prep", preprocessor),
    ("nb", GaussianNB())
])

scores = cross_val_score(model, X_rest, y_rest, cv=cv, scoring="accuracy")
print(f"Average accuracy scores for each CV fold:\n")
for i, score in enumerate(scores):
    print(f"\tFold {i+1}: {score:.4f}")

print(f"\nAverage Accuracy from CV: {scores.mean():.4f}\n")

model.fit(X_rest, y_rest)
y_pred = model.predict(X_test)

print("Test Set - Classification Report\n")
print(classification_report(y_test, y_pred))
```

Nie używam żadnych parametrów tych modeli (zrobię hiperparametryzację później)

Porównanie wyników

Przykładowy raport z wynikami (tutaj akurat do klasyfikatora Bayesa)

```
Average accuracy scores for each CV fold:

Fold 1: 0.6429
Fold 2: 0.6429
Fold 3: 0.5000
Fold 4: 0.7500
Fold 5: 0.7778
Fold 6: 0.6296
Fold 7: 0.5926
Fold 8: 0.6667

Average Accuracy from CV: 0.6503

Test Set - Classification Report
```

	precision	recall	f1-score	support
C	0.75	0.80	0.77	30
CL	0.14	0.25	0.18	4
D	0.65	0.50	0.56	22
accuracy			0.64	56
macro avg	0.51	0.52	0.51	56
weighted avg	0.67	0.64	0.65	56

Średnia dokładność (accuracy) każdego z modeli

$$\frac{TP + TN}{P + N}$$

Model	Accuracy
Klasyfikator Bayesa	0.6503
Drzewo decyzyjne	0.6635
Random Forest	0.7503
Support Vector Machine	0.5407

Największą dokładność (accuracy) na zestawach treningowych ma Random Forest, a najmniejszą Support Vector Machine, podczas gdy klasyfikator Bayesa i drzewo decyzyjne podobną. Jestem ciekawy czy Random Forest nie uległ zjawisku overfitting – przekonamy się dalej.

Zobaczmy teraz wyniki na zbiorze testowym dla poszczególnych modeli

1. Klasyfikator Bayesa

Test Set - Classification Report				
	precision	recall	f1-score	support
C	0.75	0.80	0.77	30
CL	0.14	0.25	0.18	4
D	0.65	0.50	0.56	22
accuracy			0.64	56
macro avg	0.51	0.52	0.51	56
weighted avg	0.67	0.64	0.65	56

Przetrenowany model ze sporą skutecznością wykrywa to czy pacjent przeżyje (recall 0.8), gorzej jednak sobie radzi dla tych co zmarli (recall 0.5 nie jest zadawalający, to jak rzut monetą). Najgorzej sobie radzi z przewidywaniem przeszczepu.

Model poradził sobie na zbiorze testowym prawie tak dobrze jak na treningowym, zatem nie jest przetrenowany.

2. Drzewo decyzyjne

Test Set - Classification Report				
	precision	recall	f1-score	support
C	0.67	0.67	0.67	30
CL	0.20	0.25	0.22	4
D	0.62	0.59	0.60	22
accuracy			0.61	56
macro avg	0.50	0.50	0.50	56
weighted avg	0.61	0.61	0.61	56

Model przewiduje przetrwanie trochę gorzej niż klasyfikator Bayesa, ale trochę lepiej przewiduje to, że pacjent umrze.

Dokładność jest 5% gorsza niż dla zbioru treningowego, co nas nie do końca zadawała.

3. Random Forest

Test Set - Classification Report					
	precision	recall	f1-score	support	
C	0.80	0.80	0.80	30	
CL	0.00	0.00	0.00	4	
D	0.62	0.73	0.67	22	
accuracy			0.71	56	
macro avg	0.47	0.51	0.49	56	
weighted avg	0.67	0.71	0.69	56	

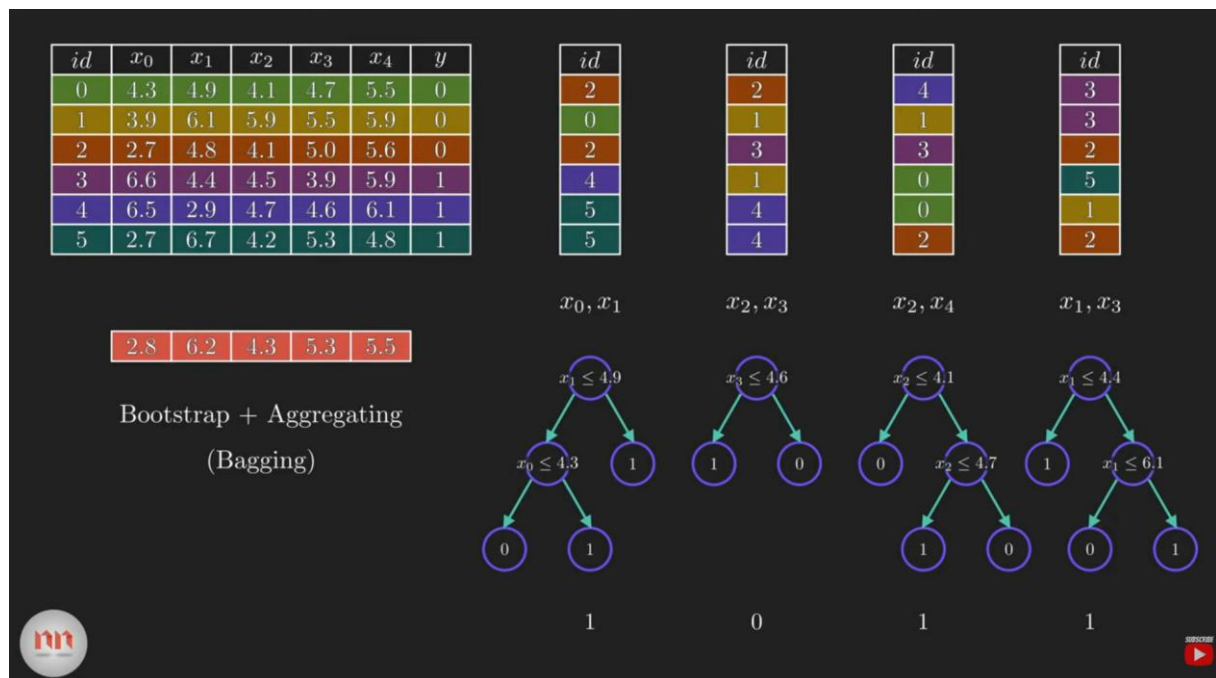
Model daje najlepsze wyniki

4. Support Vector Machine

Test Set - Classification Report					
	precision	recall	f1-score	support	
C	0.55	0.97	0.70	30	
CL	0.00	0.00	0.00	4	
D	0.67	0.09	0.16	22	
accuracy			0.55	56	
macro avg	0.40	0.35	0.29	56	
weighted avg	0.56	0.55	0.44	56	

Model prawie w bardzo słabo wykrywa śmierć pacjenta (recall na poziomie 0.09 jest śmiesznym wynikiem). Przy obecnych parametrach jest to najgorzy wynik

Krótką teoria do Random Forest



Zamiast tworzyć jedno skomplikowane drzewo decyzyjne, algorytm ten tworzy cały las niezależnych drzew. Kolejne etapy

1. **Losowanie danych (Bootstrapping / Bagging):** Każde drzewo w lesie uczy się na nieco innym, losowo wybranym podzbiorze danych treningowych.
2. **Losowanie cech:** Przy każdym podziale w drzewie algorytm wybiera losowy podzbiór cech (atrybutów), a nie wszystkie dostępne. Dzięki temu drzewa różnią się od siebie i są od siebie niezależne.
3. **Agregacja (aggregation):** Gdy model dostaje nowy przypadek do zaklasyfikowania, każde drzewo daje odpowiedź. Wynikiem końcowym lasu jest ta klasa, która zdobyła najwięcej głosów.

Porównanie metod postępowania z brakującymi wartościami

Będziemy porównywać 3 metody

- Usuwanie wierszy z brakującymi danymi (metoda zastosowana wyżej)
- Imputacja
- Interpolacja

Imputacja

Przygotowałem następujący preprocesor z imputacją danych

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("num", SimpleImputer(strategy="median"), num_cols),  
        ("cat", Pipeline([  
            ("imputer", SimpleImputer(strategy="most_frequent")),  
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))  
        ]), cat_cols),  
    ],  
)
```

Brakujące wartości typów numerycznych są zastępowane medianą, a wartości typów kategorycznych najpopularniejszą kategorią.

Interpolacja

Liczymy najbardziej pasującą wartość dla typów numerycznych

```
# Interpolation  
cat_cols = X.select_dtypes(include=["object", "str", "category"]).columns  
num_cols = X.select_dtypes(include=["number"]).columns  
X[num_cols] = X[num_cols].interpolate(method="linear", limit_direction="both")
```

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("num", SimpleImputer(strategy="median"), num_cols),  
        ("cat", Pipeline([  
            ("imputer", SimpleImputer(strategy="most_frequent")),  
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))  
        ]), cat_cols),  
    ],  
)
```

Dokładność (accuracy) w walidacji krzyżowej

Porównujemy średnią dokładność (accuracy) dla modeli i metod

Model	Usuwanie	Imputacja	Interpolacja
Klasyfikator Bayesa	0.6503	0.6619	0.2307
Drzewo decyzyjne	0.6635	0.6647	0.6502
Random Forest	0.7503	0.7630	0.7364
Support Vector Machine	0.5407	0.5810	0.5758

Imputacja daje trochę lepsze wyniki niż usuwanie brakujących danych. Ciekawy jest tutaj przypadek klasyfikatora Bayesa dla interpolacji.

Model Bayesa opiera się na matematycznych rozkładach prawdopodobieństwa. Interpolacja danych niesekwencyjnych sztucznie połączyła niezależne wiersze liniami prostymi, co stworzyło nierealne wartości pośrednie. To całkowicie zniekształciło rozkłady danych i zatarto granice między klasami, odbierając modelowi możliwość ich rozróżnienia (spadek dokładności z 0.73 do 0.23).

Modele drzewiaste (Decision Tree, Random Forest) przetrwały ten proces, ponieważ opierają się na sztywnych regułach podziału (instrukcjach warunkowych), a nie na ciągłej probabilistyce.

Ewaluacja na zbiorze testowym

Teraz zobaczymy skuteczność predykcji na zbiorze testowym

Dokładność (accuracy)			
Model	Usuwanie	Imputacja	Interpolacja
Klasyfikator Bayesa	0.64	0.67	0.20
Drzewo decyzyjne	0.61	0.60	0.64
Random Forest	0.71	0.71	0.71
Support Vector Machine	0.55	0.55	0.55

Dokładność wydaje się być taka sama dla Random Forest, w klasyfikatorze Bayesa imputacja jest najskuteczniejsza, a dla drzew decyzyjnych najlepszy wynik miała interpolacja

Czułość (recall) – format: Czułość dla tych co przetrwali Czułość dla tych co mieli przeszczep Czułość dla zmarłych			
Model	Usuwanie	Imputacja	Interpolacja
Klasyfikator Bayesa	0.67	0.79	0.09
	0.25	0.40	1.00
	0.59	0.65	0.25
Drzewo decyzyjne	0.80	0.72	0.70
	0.25	0.20	0.60
	0.73	0.47	0.57
Random Forest	0.80	0.87	0.89
	0.00	0.00	0.00
	0.73	0.59	0.56
Support Vector Machine	0.97	0.96	0.96
	0.00	0.00	0.00
	0.09	0.03	0.03

Najlepszą czułość ma Drzewo decyzyjne i Random Forest z usuniętymi brakującymi danymi.

Precyzja (precision) – format: Precision dla tych co przetrwali Precision dla tych co mieli przeszczep Precision dla zmarłych			
Model	Usuwanie	Imputacja	Interpolacja
Klasyfikator Bayesa	0.75	0.79	0.57
	0.14	0.40	0.07
	0.65	0.53	0.80
Drzewo decyzyjne	0.67	0.72	0.70
	0.20	0.20	0.50
	0.62	0.47	0.58
Random Forest	0.80	0.87	0.74
	0.00	0.00	0.00
	0.62	0.59	0.67
Support Vector Machine	0.55	0.56	0.56
	0.00	0.00	0.00
	0.57	0.33	0.33

Jeśli chodzi o precyzję wydaje się jakby Random Forest był najbardziej precyzyjny, podczas gdy klasyfikator Bayesa też nie daje najgorszych wyników dla usuwania. Odpowiedni model należy tutaj wybrać w zależności od tego czy chcemy mniej błędów popełniać przy diagnozie zdrowego pacjenta czy śmiertelnego.

F1-score – format: F1-score dla tych co przetrwali F1-score dla tych co mieli przeszczep F1-score dla zmarłych			
Model	Usuwanie	Imputacja	Interpolacja
Klasyfikator Bayesa	0.77 0.18 → 0.70 0.56	0.78 0.19 → 0.70 0.65	0.15 0.14 → 0.24 0.38
Drzewo decyzyjne	0.67 0.22 → 0.70 0.60	0.69 0.20 → 0.59 0.51	0.70 0.55 → 0.64 0.57
Random Forest	0.80 0.00 → 0.70 0.67	0.80 0.00 → 0.69 0.63	0.81 0.00 → 0.68 0.61
Support Vector Machine	0.70 0.00 → 0.70 0.16	0.70 0.00 → 0.42 0.03	0.70 0.00 → 0.42 0.06

Najlepsze wyniki tutaj daje model Random Forest. Klasyfikator Bayesa z interpolacją daje bardzo słabe wyniki ponieważ, brakujące wartości są określane na podstawie sąsiadów, co daje bardzo dużą korelację między atrybutami.

Który model i metodę wybrać?

Zależy do czego budujemy model

- Jeśli chcemy przewidzieć czy pacjent przeżyje czy umrze to Random Forest z usuwaniem (bądź imputacją) będą dobrym wyborem
- Jeśli chcemy brać pod uwagę każdą z 3 kategorii włącznie z przeszczepami, to wybrałbym Klasyfikator Bayesa z imputacją, ponieważ nie ignoruje najmniej licznej klasy

Badanie metod przetwarzania danych

Krótką analiza sugerowanych metod

1. Normalizacja - skalowanie liczby do zakresu 0-1
2. Standaryzacja – sprawia że średnia liczb to 0, a odchylenie standardowe 1
3. Dyskretyzacja – zmiana liczb na przedziały liczbowe (np. bilirubin ≤ 1 , 1-3, > 3)
4. Selekcja cech – wybranie tylko części najważniejszych cech
5. PCA – nowe cechy jako kombinacje starych, redukcja wymiarowości

Kod

1. Normalizacja

```
# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", MinMaxScaler())
        ]), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
        ]), cat_cols),
    ]
)
```

2. Standaryzacja

```
# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler())
        ]), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
        ]), cat_cols),
    ]
)
```

3. Dyskretyzacja

```
# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("num", Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", KBinsDiscretizer(n_bins=5, encode="onehot-dense", strategy="quantile"))
        ]), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
        ]), cat_cols),
    ])
)
```

4. Selekcja cech

```
# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
        ]), cat_cols),
    ])

full_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("scaler", StandardScaler()),
    ("feature_selection", SelectKBest(score_func=f_classif, k=10))
])
```

5. PCA

```
# Preprocessor
preprocessor = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), num_cols),
        ("cat", Pipeline([
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))
        ]), cat_cols),
    ])

full_pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=0.95, random_state=67))
])
```

Które modele warto przeanalizować?

Moim zdaniem nie ma sensu analizować tutaj drzew decyzyjnych i Random Forest

- Skalowanie i normalizacja nic nie dadzą ponieważ jedynie przesuną progi decyzyjne atrybutów w węzłach
- Drzewa tworzą własną dyskretyzację dla wybranych atrybutów w węzłach, robiąc dyskretyzację tylko ograniczymy wbudowaną dyskretyzację
- PCA miesza cechy, przez co mamy większą entropię, co ogranicza swobodę tworzenia drzew. Dla jednego węzła zawsze wybierana jest jedna cecha
- Drzewa mają wbudowaną selekcję cech

Przeanalizuję za to wpływ tych metod na klasyfikator Bayesa i SVM

Analiza dla klasyfikatora Bayesa

Metoda	Dokładność (accuracy)	F1-score	Uwagi
Baza	0.67	0.78 0.19 → 0.70 0.65	
Normalizacja	0.25	0.29 0.11 → 0.33 0.42	
Standaryzacja	0.26	0.33 0.12 → 0.35 0.42	
Dyskretyzacja	0.33	0.29 0.16 → 0.41 0.49	Im większe n, tym lepszy wynik, tutaj jest dla n = 5
Selekcja cech	0.14	0.00 0.10 → 0.35 0.38	
PCA	0.64	0.73 0.00 → 0.64 0.60	95%

Wyniki dla metod preprocessingowych są gorsze niż dla modelu bazowego, ponieważ zastosowane transformacje zmieniły strukturę danych w sposób niekorzystny dla klasyfikatora Bayesa. Naiwny klasyfikator Bayesa opiera się na rozkładach pojedynczych cech oraz założeniu ich warunkowej niezależności względem klasy.

- Normalizacja i standaryzacja mogły zaburzyć relacje pomiędzy wartościami cech a klasami.
- Dyskretyzacja uprościła dane, lecz jednocześnie spowodowała utratę części informacji.
- Selekcja cech prawdopodobnie usunęła cechy, które pojedynczo mogły wydawać się słabe, ale razem dostarczały istotnego sygnału klasyfikacyjnego.
- PCA zachowało większość wariancji, lecz kierunki największej wariancji nie muszą odpowiadać kierunkom najlepiej separującym klasy.

Analiza dla Support Vector Machine (SVM)

Metoda	Dokładność (accuracy)	F1-score	Uwagi
Baza	0.55	0.70 0.00 → 0.42 0.06	
Normalizacja	0.70	0.77 0.00 → 0.68 0.65	
Standaryzacja	0.74	0.80 0.00 → 0.71 0.70	
Dyskretyzacja	0.76	0.82 0.00 → 0.73 0.72	n = 5
Selekcja cech	0.70	0.77 0.00 → 0.68 0.65	
PCA	0.68	0.75 0.00 → 0.65 0.61	95%

W przypadku klasyfikatora SVM zastosowanie metod preprocessingu wyraźnie poprawiło wyniki względem modelu bazowego. Jest to zgodne z charakterystyką SVM, ponieważ algorytm ten jest wrażliwy na skalę cech i sposób reprezentacji danych.

- Normalizacja oraz standaryzacja poprawiły wyniki, ponieważ ograniczyły dominację cech o dużych zakresach wartości i umożliwiły lepsze wyznaczenie granicy decyzyjnej.
- Najlepszy wynik uzyskano dla dyskretyzacji przy liczbie przedziałów $n = 5$, co może oznaczać, że uproszczenie wartości ciągłych do przedziałów zmniejszyło wpływ szumu i lepiej uchwyciło progowy charakter zależności w danych.
- PCA oraz selekcja cech również poprawiły wynik względem bazy, jednak były słabsze od standaryzacji i dyskretyzacji, ponieważ mogły usuwać część informacji istotnych dla separacji klas.

Dyskretyzacja z SVM dała lepszy wynik niż najlepsze modele bez przetwarzania danych (Random Forest z usuwaniem danych i klasyfikator Bayesa z imputacją). Chcę jeszcze spróbować Standaryzację / Normalizację + Dyskretyzację z SVM.

Dla standaryzacji + dyskretyzacji z SVM nie ma poprawy

Test Set - Classification Report					
	precision	recall	f1-score	support	
C	0.74	0.91	0.82	47	
CL	0.00	0.00	0.00	5	
D	0.81	0.66	0.72	32	
accuracy			0.76	84	
macro avg	0.52	0.52	0.51	84	
weighted avg	0.72	0.76	0.73	84	

Dla normalizacji + dyskretyzacji z SVM też nie ma poprawy

Test Set - Classification Report					
	precision	recall	f1-score	support	
C	0.74	0.91	0.82	47	
CL	0.00	0.00	0.00	5	
D	0.81	0.66	0.72	32	
accuracy			0.76	84	
macro avg	0.52	0.52	0.51	84	
weighted avg	0.72	0.76	0.73	84	

Hiperparametryzacja

Klasyfikator Bayesa

```
model = Pipeline([
    ("prep", preprocessor),
    ("nb", GaussianNB())
])

param_grid = {
    # GaussianNB
    'nb__var_smoothing': np.logspace(0, -9, num=100),
    # Preprocessing
    'prep__num_strategy': ['mean', 'median'],
    'prep__cat_imputer_strategy': ['most_frequent']
}

grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1, # All available cores
    # verbose=1
)
```

```
print("Best params")
for param_name in sorted(param_grid.keys()):
    print(f"\t{param_name}: {grid_search.best_params_[param_name]}")
print(f"\nBest Accuracy from cv: {grid_search.best_score_:.4f}\n")

best_model = grid_search.best_estimator_
y_pred_best = best_model.predict(X_test)

print("Test Set (Zoptymalizowany Model) - Classification Report\n")
print(classification_report(y_test, y_pred_best))
```

Używamy tutaj klasyfikator Bayesa z imputerem

	Accuracy	F1-score	Smoothing	Cat strat	Num strat
Parametryzacja	0.67	0.62	1e-9	most freq	median
Parametryzacja	0.65	0.64	1e-7.5476	most freq	mean

Porównywalne wyniki po parametryzacji

Drzewo decyzyjne

```
param_grid = {  
    # DecisionTreeClassifier  
    'dt__max_depth': [3, 4, 5, 6],  
    'dt__min_samples_split': [2, 5, 10, 20],  
    'dt__min_samples_leaf': [1, 2, 4, 6],  
    'dt__criterion': ['gini', 'entropy']  
}
```

	Accuracy	F1-score	Max depth	Min samples split	Min samples leaf	Criterion
Domyślnie	0.61	0.61	None	2	1	gini
Parametryzacja	0.67	0.64	3	2	1	entropy

Lepsze wyniki na zbiorze testowym po parametryzacji

Random Forest

```
param_grid = {  
    # RandomForestClassifier  
    'rt__n_estimators': [50, 100, 200],  
    'rt__max_depth': [3, 5, 7, None],  
    'rt__min_samples_split': [2, 3, 4],  
    'rt__min_samples_leaf': [1, 2, 4],  
    'rt__max_features': ['sqrt', 'log2'],  
    'rt__criterion': ['gini', 'entropy']  
}
```

	Accuracy	F1-score	Estimators	Max depth	Min samples split	Min samples leaf	Max features	Criterion
Domyślnie	0.71	0.69	100	None	2	1	sqrt	gini
Parametryzacja	0.73	0.70	100	7	3	1	Log2	entropy

Lepsze wyniki na zbiorze testowym po parametryzacji. Parametryzacja trwała aż 2 min 11 sekund.

SVM

```
param_grid = {  
    # Preprocessor  
    'prep__num__scaler__n_bins': [3, 5, 7],  
    'prep__num__scaler__strategy': ['uniform', 'quantile'],  
  
    # SVM  
    'svc__C': [0.1, 1, 10, 100],  
    'svc__kernel': ['rbf', 'linear', 'sigmoid'],  
    'svc__gamma': ['scale', 'auto']  
}
```

SVM + dyskretyzacja

	Accuracy	F1-score	N bins	Scaler strat	C	Gamma	Kernel
Domyślnie	0.76	0.73	5	quantile	1	scale	rbf
Parametryzacja	0.76	0.73	5	quantile	1	scale	rbf

Dokładnie te same parametry wygrały, co były używane od początku.

Zastosowane metody łagodzenia przeuczenia

1. Smoothing dla klasyfikatora Bayesa

$$P(w|c) = \frac{N_{w,c} + 1}{N_c + |V|}$$

2. Regularyzacja w SVM (parametr C jako λ)

L1 Regularization

$$\text{Cost} = \sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2 + \lambda \sum_{j=0}^M |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^N (y_i - \sum_{j=0}^M x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^M W_j^2}_{\text{Regularization Term}}$$

Biblioteki

- **scikit-learn**

Biblioteka używana do budowy, trenowania i oceny modeli uczenia maszynowego. W projekcie wykorzystano ją m.in. do klasyfikacji, walidacji krzyżowej, przetwarzania danych i strojenia hiperparametrów.

- **NumPy**

Biblioteka wspierająca obliczenia numeryczne w Pythonie. Służy głównie do pracy z tablicami i wykonywania szybkich operacji matematycznych na danych.

- **pandas**

Biblioteka do analizy i przetwarzania danych tabelarycznych. W projekcie służy do przechowywania danych w postaci ramek danych oraz ich czyszczenia i przygotowania.

- **ucimlrepo**

Biblioteka umożliwiająca pobieranie zbiorów danych z repozytorium UCI Machine Learning Repository. W projekcie wykorzystano ją do załadowania badanego zbioru danych.

- **matplotlib**

Biblioteka do tworzenia wykresów i wizualizacji danych. Została użyta głównie na etapie eksploracyjnej analizy danych.

- **seaborn**

Biblioteka do estetycznej wizualizacji danych oparta na matplotlib. Ułatwia tworzenie czytelnych wykresów statystycznych, np. rozkładów i zależności między cechami.

Bibliografia

<https://www.youtube.com/watch?v=v6VJ2RO66Ag>

https://www.youtube.com/watch?v=_YPScrckx28